

```
legacy_code = read_java_file(input_file)
modernized_code = modernize_java_code(legacy_code,
task="upgrade") # or "springboot"
write_java_file(output_file, modernized_code)
print(f"Modernized code written to {output_file}")
```

Java modernisation in action

Let's walk through a realistic legacy Java example to see how this works.

Suppose your file *legacy_code.java* contains:

```
public class Rectangle {
private int length;
private int width;
public Rectangle(int length, int width) {
this.length = length;
this.width = width;
}
public int getArea() {
return length * width;
}
}
```

The script generates a prompt like:

You are an expert Java developer. Modernize the following Java code to conform to Java 17 standards, improving readability, performance, and using new language features where applicable. Please return only the updated Java code.

Legacy Java code is:

```
public class Rectangle {
private int length;
private int width;
public Rectangle(int length, int width) {
this.length = length;
this.width = width;
}
public int getArea() {
return length * width;
}
}
```

Here's an example of how OpenAI may respond:

```
public record Rectangle(int length, int width) {
public int getArea() {
return length * width;
}
}
```

Notice how the code now uses the record feature introduced in Java 16, streamlining the class definition.

Let's now take a Spring Boot migration example.

Servlet-based legacy code is:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class HelloServlet extends HttpServlet {
public void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
response.setContentType("text/html");
PrintWriter out = response.getWriter();
out.println("Hello, World!");
}
}
```

The script, with *task="springboot"*, constructs the relevant prompt and gets this possible OpenAI output:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.
SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.
RestController;
@SpringBootApplication
public class HelloApplication {
public static void main(String[] args) {
SpringApplication.run(HelloApplication.class, args);
}
}
@RestController
class HelloController {
@GetMapping("/")
public String hello() {
return "Hello, World!";
}
}
```

The servlet is now modernised into a concise, annotation-driven Spring Boot application.

Best practices for prompt engineering

To get optimal results, keep the following in mind:

- **Be specific:** Clearly state the modernisation task and expectations.
- **Limit output scope:** Ask the model to return code only, without extra commentary.
- **Iterate:** Experiment with prompt phrasing and refine for better results.